LAPORAN PRAKTIKUM BASIS DATA

PERTEMUAN 2 — HAK AKSES PADA POSTGRESQL

Role-Based Access Control (RBAC) • GRANT • REVOKE

Nama Mahasiswa	Riza Zaelani Dwi P
NIM	2507054
Mata Kuliah	Praktikum RABD

A. Tujuan Praktikum

Mengonfigurasi hak akses berbasis role pada PostgreSQL, membuat role dan user, menerapkan prinsip least privilege, membatasi akses pada tingkat tabel dan kolom, serta menguji bahwa operasi yang tidak diizinkan ditolak oleh database.

B. Skenario Praktikum

Skenario mengikuti tugas terstruktur pada halaman 27–28 modul Pertemuan 2: Sistem Tata Usaha dan Keuangan Akademik. Dua role dibuat, yaitu role_keuangan dan role_tata_usaha. Role mahasiswa juga diperbarui agar dapat membaca tagihan kuliah.

C. Struktur Tabel

Tabel yang digunakan: mahasiswa, mata_kuliah, nilai_akademik, dan tagihan_kuliah.

Tabel tagihan_kuliah memiliki kolom id_tagihan, nim, nominal, status_lunas, dan tanggal_jatuh_tempo. Kolom nim merujuk ke mahasiswa(nim); nominal tidak boleh negatif; status_lunas bernilai boolean.

D. Konfigurasi Hak Akses

Role	Hak akses	Pembatasan
role_keuangan	SELECT pada tabel di schema public; INSERT pada tagihan_kuliah; UPDATE hanya kolom status_lunas; USAGE pada sequence tagihan	Tidak boleh UPDATE kolom nominal atau nim.
role_tata_usaha	SELECT, INSERT, UPDATE pada mahasiswa dan mata_kuliah	Tidak diberi akses ke tagihan_kuliah maupun nilai_akademik.
role_mahasiswa	SELECT pada tagihan_kuliah (ditambahkan)	Akses baca saja untuk tabel tagihan.

E. Skrip SQL

Skrip lengkap tersedia pada file Tugas_Pertemuan_2_Hak_Akses_PostgreSQL.sql. Berikut bagian inti konfigurasi:

```
-- Jalankan sebagai administrator PostgreSQL pada database akademik

CREATE TABLE IF NOT EXISTS tagihan_kuliah (
    id_tagihan SERIAL PRIMARY KEY,
    nim VARCHAR(10) REFERENCES mahasiswa(nim),
    nominal NUMERIC(12,2) NOT NULL CHECK (nominal >= 0),
    status_lunas BOOLEAN DEFAULT FALSE,
    tanggal_jatuh_tempo DATE NOT NULL
);

CREATE ROLE role_keuangan;
```

```
CREATE ROLE role_tata_usaha;

GRANT USAGE ON SCHEMA public TO role_keuangan, role_tata_usaha;

GRANT SELECT ON ALL TABLES IN SCHEMA public TO role_keuangan;
GRANT INSERT ON TABLE tagihan_kuliah TO role_keuangan;
GRANT UPDATE (status_lunas) ON TABLE tagihan_kuliah TO role_keuangan;
GRANT USAGE, SELECT ON SEQUENCE tagihan_kuliah_id_tagihan_seq TO role_keuangan;

GRANT SELECT, INSERT, UPDATE ON TABLE mahasiswa, mata_kuliah TO role_tata_usaha;
REVOKE ALL PRIVILEGES ON TABLE tagihan_kuliah, nilai_akademik FROM role_tata_usaha;

GRANT SELECT ON TABLE tagihan_kuliah TO role_mahasiswa;

CREATE USER staff_keuangan01 WITH PASSWORD '[GANTI_DENGAN_PASSWORD_KUAT]';
CREATE USER staff_tu01 WITH PASSWORD '[GANTI_DENGAN_PASSWORD_KUAT]';
GRANT role_keuangan TO staff_keuangan01;
GRANT role_tata_usaha TO staff_tu01;
```

F. Pengujian dan Hasil yang Diharapkan

1. Verifikasi hak akses

Jalankan sebagai administrator:

```
SELECT grantee, table_name, privilege_type
FROM information_schema.role_table_grants
WHERE grantee IN ('role_keuangan','role_tata_usaha','role_mahasiswa')
ORDER BY grantee, table_name, privilege_type;
```

Selain itu, jalankan \du untuk melihat daftar role dan keanggotaan, serta periksa sequence tagihan_kuliah_id_tagihan_seq.

2. Uji INSERT tagihan oleh staf keuangan

```
SET ROLE staff_keuangan01;
INSERT INTO tagihan_kuliah (nim, nominal, status_lunas, tanggal_jatuh_tempo)
VALUES ('2024001', 1500000.00, FALSE, '2026-12-31');
RESET ROLE;
```

Hasil yang diharapkan: INSERT berhasil jika mahasiswa dengan NIM 2024001 tersedia dan user memiliki hak INSERT serta hak sequence.

3. Uji UPDATE status_lunas (diizinkan)

```
SET ROLE staff_keuangan01;
UPDATE tagihan_kuliah SET status_lunas = TRUE WHERE nim = '2024001';
RESET ROLE;
```

Hasil yang diharapkan: UPDATE pada kolom status_lunas diizinkan.

4. Uji UPDATE nominal (harus ditolak)

```
SET ROLE staff_keuangan01;
UPDATE tagihan_kuliah SET nominal = 1000 WHERE nim = '2024001';
RESET ROLE;
```

Hasil yang diharapkan: PostgreSQL menolak query dengan pesan permission denied untuk tabel/kolom karena role_keuangan hanya diberi UPDATE pada kolom status_lunas. Jalankan uji ini secara terpisah; setelah error, reset atau buka sesi baru jika diperlukan.

5. Uji SELECT tagihan oleh staf tata usaha (harus ditolak)

```
SET ROLE staff_tu01;
SELECT * FROM tagihan_kuliah;
RESET ROLE;
```

Hasil yang diharapkan: PostgreSQL menolak query dengan pesan permission denied for table tagihan_kuliah.

G. Analisis

Pengaturan hak akses menunjukkan prinsip least privilege. Staf keuangan dapat membaca data yang diperlukan, menambahkan tagihan, dan memperbarui status pelunasan, tetapi tidak dapat mengubah nominal atau NIM setelah data dimasukkan. Staf tata usaha dapat mengelola data mahasiswa dan mata kuliah, tetapi tidak dapat membaca tagihan kuliah maupun nilai akademik. Dengan demikian, tanggung jawab dipisahkan berdasarkan fungsi kerja.

Role mahasiswa diberikan hak SELECT pada tagihan_kuliah agar dapat membaca informasi tagihan. Hak tersebut tidak memberikan izin untuk menambah atau mengubah tagihan.

H. Kesimpulan

Role-Based Access Control memudahkan pengelolaan hak akses karena izin ditetapkan pada role dan kemudian diwariskan kepada user. PostgreSQL mendukung pembatasan akses pada tingkat tabel dan kolom. Pengujian terhadap operasi yang diizinkan dan ditolak diperlukan untuk membuktikan bahwa konfigurasi sesuai dengan kebijakan keamanan. Prinsip least privilege membantu mencegah akses atau perubahan data di luar tugas pengguna.

I. Bukti yang Harus Ditambahkan

☐ Screenshot pembuatan tabel dan role.

☐ Screenshot GRANT dan penetapan role kepada user.

☐ Screenshot INSERT tagihan yang berhasil.

☐ Screenshot UPDATE status_lunas yang berhasil.

☐ Screenshot error saat staff_keuangan01 mencoba UPDATE nominal.

☐ Screenshot error saat staff_tu01 mencoba SELECT tagihan_kuliah.

☐ Screenshot hasil pemeriksaan privilege/role.

Catatan: Hasil di atas adalah hasil yang diharapkan dari konfigurasi, bukan log eksekusi aktual. Tambahkan screenshot dari DBeaver setelah skrip dijalankan di server PostgreSQL.